Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

Implementar sync_car.c em C com pthreads, a sincronização do embarque de passageiros nos vagões do metrô da METROREC. Defini a struct station e as funções station_init, station_load_car, station_wait_for_car e station_on_board, garantindo que o carro só parta com todos os passageiros sentados e com o vagão cheio ou sem ninguém esperando, permitindo que vários passageiros embarquem ao mesmo tempo, usando um único mutex por estação e sem busy-wait. A validação foi feita com o sync_car-runner.c.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Uso das funções pthread listadas (item 1)	OK	grep pthread_ sync_car.c
Sem semáforos ou outras primitivas (item 2)	OK	grep -E "sem_|spin" sync_car.c (sem resultado)
Um único lock por struct station (item 3)	OK	sync_car.h: um só pthread_mutex_t
Embarque simultâneo (item 5)	OK	Reserva feita em wait_for_car; runner atrasa os on_board e passa
Sem busy wait (item 6)	OK	todo bloqueio em pthread_cond_wait dentro de while
makefile gera sync_car e limpa (item 7)	OK	make clean && make
Carro parte cheio ou sem ninguém esperando, todos sentados	OK	./sync_car
Estação vazia e carro com count = 0	OK	Testes iniciais do runner (alarmes de 1s e 2 s)
Repetição (resultado não depende da ordem das threads)	OK	200 execuções: ok=200 falhas=0 (evidencias.log)
Sem condição de corrida no monitor	OK	ThreadSanitizer: nenhum aviso em sync_car.c (Seção 5.2)
README	OK	README.md
Relatório	OK	Esse documento

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
```

Repetição (concorrência muda a cada execução):

```
for i in $(seq 200); do ./sync_car > /dev/null || echo "falhou na $i"; done; echo fim
```

Saída esperada: Looks good! na execução única, e apenas "fim" na repetição.

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **sync_car.h → struct station (mutex, cond_chegou_carro, cond_todos_sentados, esperando, assentos_livres, em_transito) e prototipos.**
2. **sync_car.c → station_init, station_load_car, station_wait_for_car e station_on_board.**
3. **sync_car-runner.c → programa de teste fornecido pelo professor, sem alterações (contém o main).**
4. **Makefile → gera o executavel sync_car e o alvo clean.**
5. **README.md → arquivos, como compilar, executar e testar e o SO.**
6. **evidencias.log → log automatico das sessões no terminal.**

Decisões técnicas:

1. **struct station definida no sync_car.h, e não só no .c → o runner declara struct station station; na pilha do main e precisa saber o tamanho da struct → sem isso o runner não compila.**
2. **Reserva do assento dentro de station_wait_for_car (esperando--, assentos_livres--, em_transito++ na mesma seção crítica) → o item 5 exige que wait_for_car retorne antes de qualquer on_board, reservar só no on_board deixaria vários passageiros verem a mesma vaga → sem overbooking.**
3. **esperando++ antes do while do passageiro → o carro precisa enxergar quem está dormindo → evita o carro partir vazio com gente na plataforma.**
4. **Condição de partida: (assentos_livres == 0 OU esperando == 0) E em_transito == 0 → cobre vagão cheio, ninguém esperando e todos sentados → o carro dorme na negação dessa condição.**
5. **broadcast na chegada do carro e signal no on_board só quando em_transito chega a 0 → podem caber vários passageiros, mas só existe um carro, e só a saída do último do corredor pode liberar a partida → custo: o broadcast acorda todos os que esperam, mesmo os que não vão caber.**
6. **Zerar assentos_livres ao partir, e usar assentos_livres == 0 também como "sem carro" → ninguém embarca num carro que já saiu → dispensa uma variável carro_presente.**
7. **while em volta de todo pthread_cond_wait → acordar não garante que a condição é verdadeira → cada thread recheca o estado ao acordar.**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Estudo e modelagem	leitura do enunciado e do runner, modelagem do monitor em pseudocódigo antes do C: 3 contadores, 2 variáveis de condição, quem dorme e quem acorda cada uma, e a condição de partida.	Não houve
S2	Base do projeto	Repositório, .gitignore, runner, sync_car.h e station_init.	Não houve
S3	Uma função por commit	Tradução do pseudocódigo função a função, compilando só o monitor com gcc -Wall -Wextra -c (o runner só linka quando as 4 existem).	Não houve

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S4	Verificação	Runner, repetição em massa (200 execuções) e ThreadSanitizer.	Não houve

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Modelagem do monitor em pseudocódigo (as 4 funções) a partir do enunciado e do runner	Sucesso		30/10	—
2	S2	Repositório, .gitignore e runner do professor	Sucesso		31/10 17:13–17:24	commits c147b48, 071b330, e09b79d
3	S2	Primeira versão de station_init	Falha corrigida	pthread_mutex_init repetido e só uma das duas conds inicializada	01/11 13:27	evidencias.log 13:27 (cat sync_car.c)
4	S2	struct station no sync_car.h e station_init corrigida	Sucesso		01/11 13:36	commit 7b5754b
5	S3	station_on_board	Sucesso		01/11 13:51	commit 277bd39
6	S3	station_wait_for_car	Sucesso		01/11 13:57	commit d4eebd5
7	S3	Função do carro escrita com o nome station_on_board e int count alterei o .h para o erro sumir	Falha corrigida	conflicting types / unused parameter count: o contrato do .h é ditado pelo runner renomeei para station_load_car e restaurei o .h	01/11 15:06	Print 1
8	S3	station_load_car final	Sucesso		01/11 15:10	commit 6ff8eef
9	S3	.gitignore não ignorava sync_car.o	Falha corrigida	Linhas com espaços no início (coladas pelo site do GitHub)	01/11 15:11	Print 4, commit 29aa0f6
10	S4	Makefile e primeira	Sucesso		01/11 ~15:14	Print 2, commit 1c9fd7e

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
		execução do runner				
11	S4	ThreadSanitizer (-fsanitize=thread)	Sucesso	Saída 66 e 5 avisos, todos em sync_car-runner.c (volatile sem atômico e threads sem join) nenhum em sync_car.c	01/11 sessão 15:27	evidencias.log (tsan, grep WARNING)
12	S4	200 execuções do runner	Sucesso		01/11 sessão 15:27	evidencias.log (ok=200 falhas=0), commit ab56a85
13	S4	README e geração do lhass.tar	Sucesso		02/11 17:03–19:55	commits 033985f, e2648fb
14	S4	Caso inválido carro com count = -1 e 5 passageiros esperando	Sucesso		04/11 17:55	Print 5

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início das sessões de trabalho, rodei:

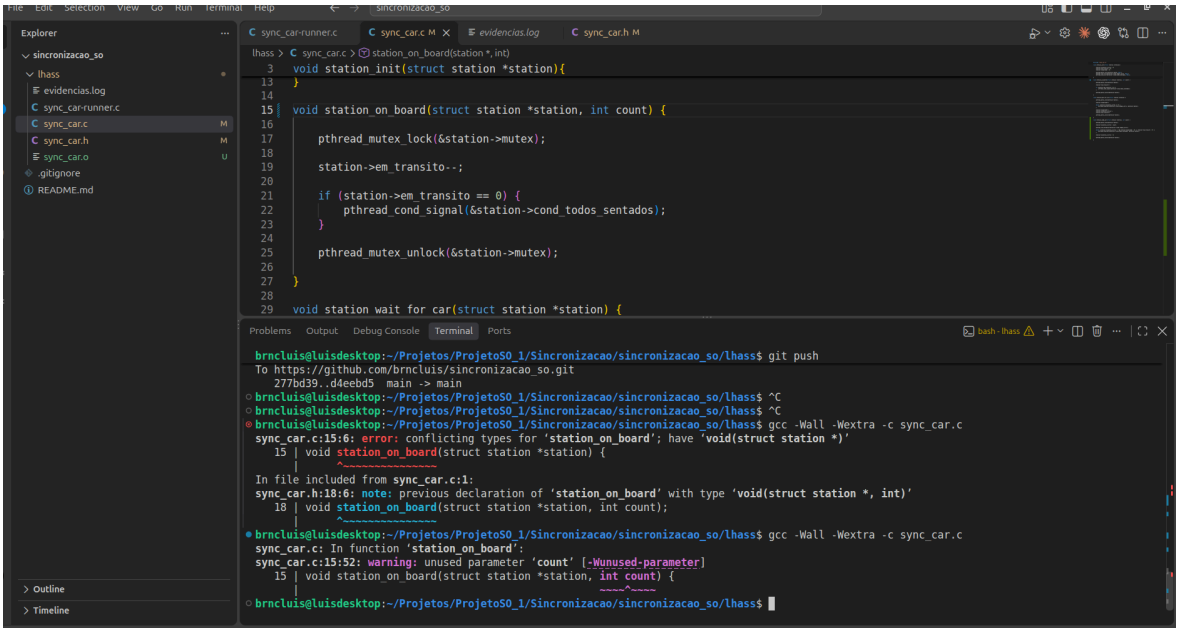
```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

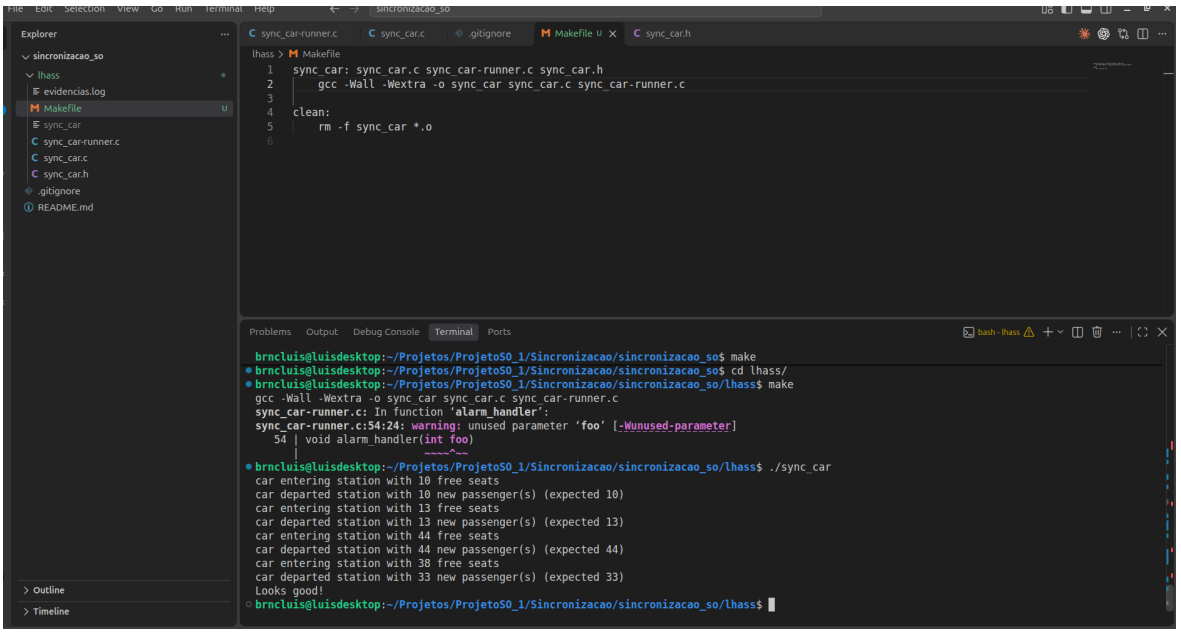
- Erro principal: o momento do bug mais relevante da semana.

Print 1: conflicting types em station_on_board, depois unused parameter count (tentativa 7).

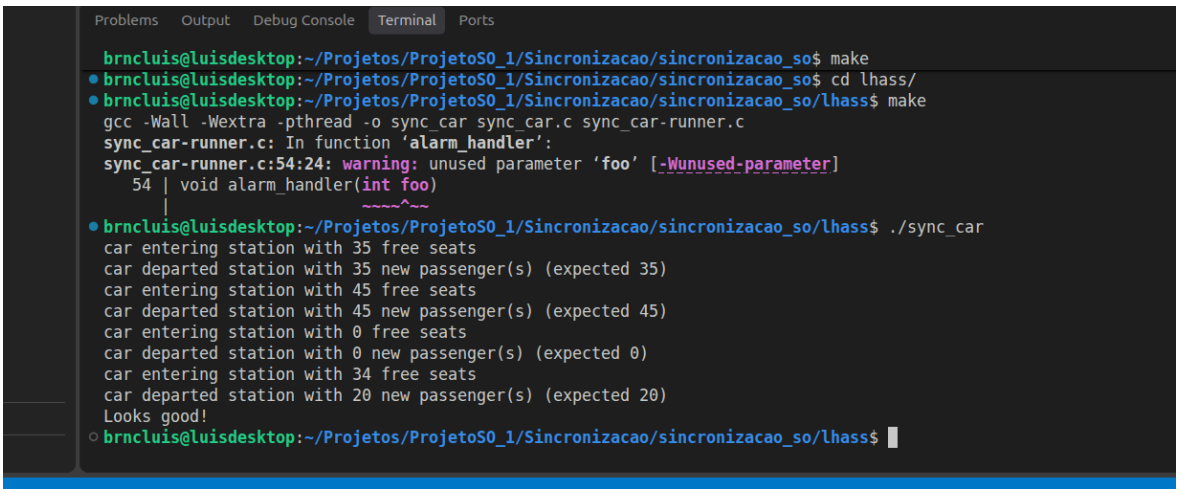


- Validação final: teste passando / saída correta.

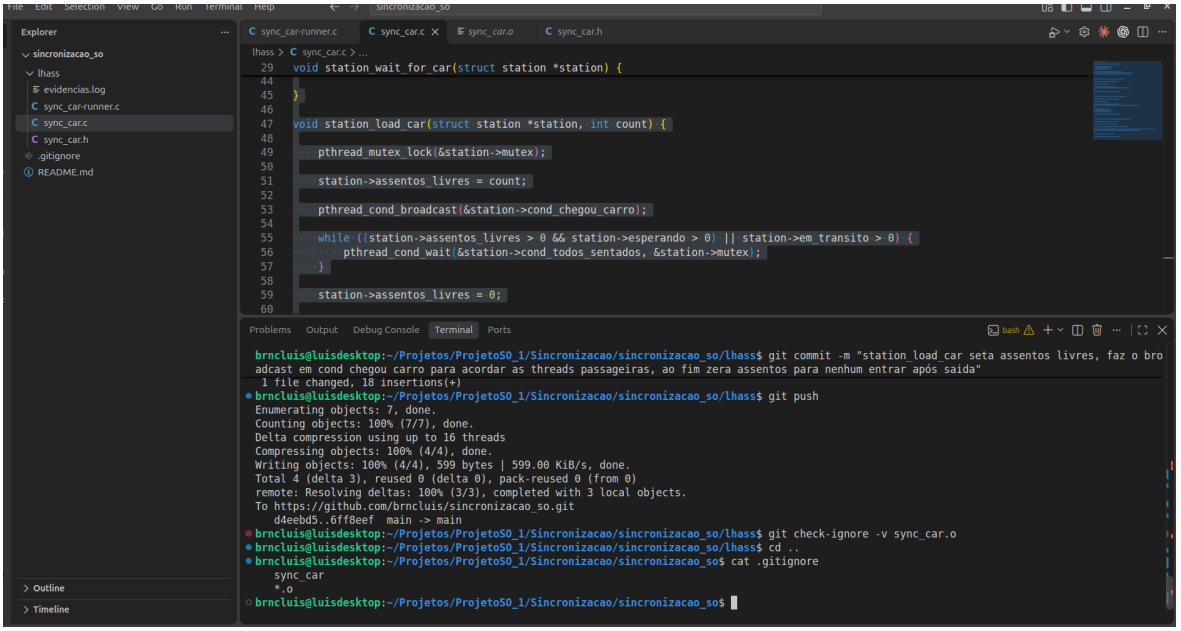
Print 2: Makefile, make e runner terminando em Looks good! (tentativa 10).



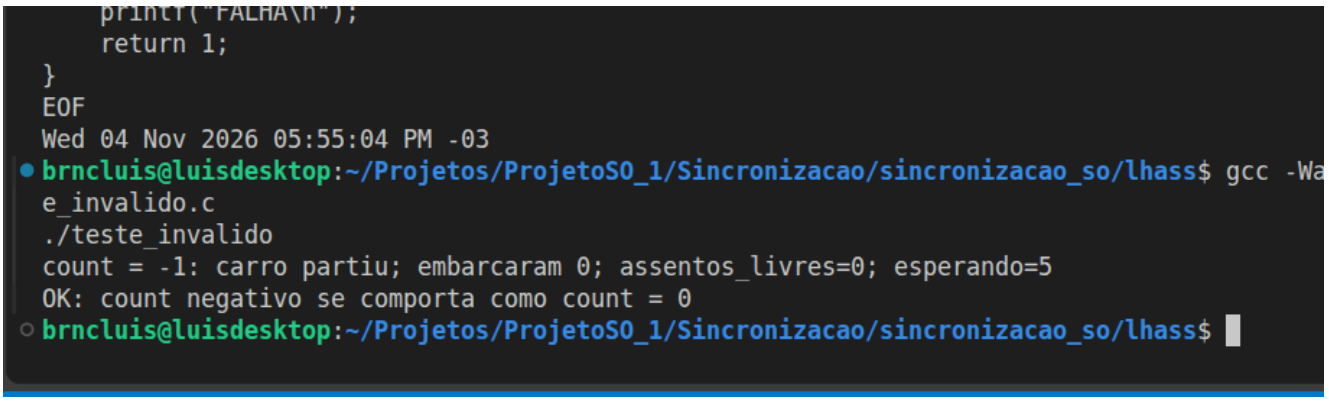
Print 3: build e execução da versão final (02/11), um carro com 0 vagas parte sem embarques e o último sai com 14 vagas sobrando porque ninguém mais esperava (Looks good!).



Print 4: cat .gitignore mostrando as linhas recuadas (tentativa 9).



Print 5: Caso inválido carro com count = -1 e 5 passageiros. (tentativa 14)



7. Uso de IA

Assim como nos projetos anteriores, usei a IA como apoio de estudo: para relembrar funções e sintaxe da biblioteca pthread (pthread_cond_wait com o mutex, signal e broadcast, inicialização) e para visualizar, em diagramas, o modelo que eu tinha em mente (quem dorme em cada variável de condição e quem acorda

quem). Também usei para revisar as funções que escrevi, apontando erros que corrigi antes do commit (& faltando no pthread_cond_init, lock na variável de condição em vez do mutex, broadcast dentro do while, nome errado da station_load_car) e para entender o runner e a saída do ThreadSanitizer. Assim como auxílio nesse documento e no README.

Prompts principais (liste, sem história):

"Me explique o que esse código do professor faz em paralelo ao que desenhei em pseudocódigo", "Verifique o .tar e rode a bateria de testes", "Gere um script com testes".

O que validei manualmente e como:

O pseudocódigo das 4 funções foi construído por mim, as funções em C foram feitas por mim. Rodei o runner, as 200 execuções e o ThreadSanitizer no meu terminal (evidencias.log).

8. Reflexão Final (5–8 linhas)

Reflexão:

O que mais aprendi neste projeto foi modelar o problema antes de escrever código. Desenhei o monitor em pseudocódigo (três contadores, duas variáveis de condição, quem dorme e quem acorda cada uma) e só depois fui pra o C, uma função por commit. Isso fez cada função ficar curta e fácil de verificar. Aprendi também que uma execução que passa não prova nada em concorrência, por isso rodei 200 vezes e usei o ThreadSanitizer. Por último, na próxima vez eu executaria mais testes próprios desde o início e não só os do professor.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Provavelmente executaria mais testes para fazer uma checagem mais completa e acordar só quantos cabem, com um signal por vaga, em vez de um broadcast para todos.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.